Hi

# VB時代は業務ドリブンだった Web時代に失われたもの

H A R M O N I C *i n s i g h t*

HARMONIC insight — Cloud vs Local シリーズ #2

# VB時代 — 業務が出発点だった

### 開発の流れ

業務要件 → 画面設計 → 実装（VB / Delphi / Access）

### 特徴

業務担当者が画面を決める
技術選定は手段
市民開発者が成立していた

### 結果

経理の人が自分でマクロを書く
現場主導の改善サイクル
業務に直結した開発

# Web時代 — アーキテクチャが出発点に

### 開発の流れ

技術選定 → アーキテクチャ設計 → API設計 → DB設計 → インフラ → … → 業務要件はどこ？

### 議論の9割が技術の話

React か Next か Nuxt か？ SPA か SSR か？
REST か GraphQL か？ マイクロサービスか？
コンテナは？ サーバーレスは？ CI/CDは？
→ 業務担当者は完全に蚊帳の外

**「要件定義書ください」で業務との間に壁ができた**

# 業務に到達するまでの距離

### VB時代

業務 → コード → 動く
たった3ステップ

### 今のWeb開発

業務 → 要件定義 → API設計 → DB設計
→ インフラ → フロント → 状態管理
→ CI/CD → やっと動く

### 本質的な問題

Webは汎用プラットフォーム。業務に到達するまでの抽象レイヤーが多すぎる。
VBは「フォームにボタンを置いてダブルクリックしたらコードを書く」で業務ロジックに直結していた。

# 業務ドリブンへ回帰する

**Insight シリーズのアプローチ**

WPF + Syncfusion で Officeファイルを「中から」直接操作
AI で業務ロジックを自然言語化
Python スクリプト — VBマクロの現代版

→ 「アーキテクチャのために業務を犠牲にする」のではなく
「業務のためにアーキテクチャを選ぶ」

**原点回帰 — 業務が先、技術は手段**

HI

# ありがとうございました

H A R M O N I C *i n s i g h t*

h-insight.jp